Guion del video: 10 minutos, 2 integrantes

Tiempo	Integrante	Criterios de la rúbrica	Peso
0:00 – 4:30	**Integrante 1**	Arquitectura + Servicio desplegado + Infraestructura como Código	10 % + 10 % + 25 %
4:30 – 10:00	**Integrante 2**	Pipeline CI/CD + Desempeño y monitoreo	25 % + 30 %

Reglas del enunciado que cumple este guion:

- Cada integrante **muestra su propia cuenta de AWS Academy** al empezar (Learner Lab con su nombre e indicador **AWS** ● verde; luego la consola con `voclabs/...` y el ID de cuenta arriba a la derecha).
- Cada integrante **ejecuta comandos en vivo** en su cuenta. Nadie solo narra.
- Los dos tienen la misma infraestructura desplegada en su propia cuenta, con el mismo código Terraform.

---

1. Verificación de la rúbrica (nivel "Excelente", 3 puntos)

Revisé cada exigencia del nivel "Excelente" contra el proyecto. La columna **Verificado** indica qué se probó antes de entregarles el proyecto. La columna **En el video** indica dónde lo ve el docente.

Arquitectura (10 %)

Exigencia "Excelente"	Cómo lo cumple el proyecto	Verificado	En el video
Explica la arquitectura	Diagrama con iconos oficiales AWS + discurso del integrante 1	Diagrama revisado visualmente	0:05
Segmentación pública/privada	6 subredes: 2 públicas (ALB, NAT), 2 privadas de app (EC2), 2 privadas de BD sin ruta a Internet	`terraform test` comprueba las subredes y sus zonas	0:05 y consola VPC
Balanceo	ALB con health check `/health` que reparte entre instancias	Prueba local con 2 instancias: reparto 50/50	0:05 y 2:00
Alta disponibilidad	2 zonas, ASG mín. 2, reemplazo automático, réplica en otra zona	Configuración validada con `terraform test`	0:05 y **4:08 (demostración en vivo)**
Seguridad	SG-ALB → SG-App → SG-DB encadenados; sin IP pública; IMDSv2; RDS cifrado	Reglas revisadas en `main.tf`	0:05 y consola SG
Justifica decisiones	Tabla de justificaciones (sección 2), incorporada al discurso	—	0:05

Exigencia "Excelente"	Cómo lo cumple el proyecto	Verificado	En el video
Diagrama completo y consistente	Usuarios, VPC, subredes, ALB, EC2, SG, RDS, S3, CI/CD y flujo de tráfico, con los mismos nombres y CIDR que Terraform	Comparado con `main.tf` y `variables.tf`	0:05

Infraestructura como Código (25 %)

Exigencia "Excelente"	Cómo lo cumple el proyecto	Verificado	En el video
Completamente automatizada	Terraform crea los 46 recursos; nada se crea a mano (solo se usa el `LabInstanceProfile` que ya trae Academy)	`terraform test` con proveedor simulado: crea todo sin errores	2:40
Organizada correctamente	`provider.tf`, `variables.tf`, `main.tf`, `outputs.tf`, `terraform.tfvars.example`, `user_data.sh`	`terraform fmt` y `validate` sin errores	2:40
init, plan, apply y output	Se ejecutan los cuatro en vivo; el `apply` hace un cambio real (versión)	`user_data.sh` renderizado y validado con `bash -n`	2:40 – 4:08

Servicio desplegado (10 %)

Exigencia "Excelente"	Cómo lo cumple el proyecto	Verificado	En el video
Funciona en AWS con endpoints válidos	`/health`, `/status`, `/api/info`, `/api/visits` (escribe en la primaria, lee de la réplica)	15 pruebas unitarias + 4 de integración con MySQL real, todas aprobadas	2:00
Validado con navegador o curl	Navegador (cambia la instancia al recargar) + 4 comandos `curl`	Comandos probados localmente	2:00

Pipeline CI/CD (25 %)

Exigencia "Excelente"	Cómo lo cumple el proyecto	Verificado	En el video
Validaciones	flake8 + `terraform fmt` + `terraform validate`	Todo pasa localmente; workflow validado con `actionlint`	4:47
Pruebas	15 unitarias (etapa 1) + 4 de integración contra el ALB (etapa 4)	Ambas suites aprobadas localmente	4:47
Despliegue / preparación	`terraform plan` (etapa 3) + instance refresh del ASG (etapa 4)	Sintaxis validada con `actionlint`	4:47
Integración adecuada	Las pruebas de integración corren contra el balanceador real después del despliegue	—	4:47

Exigencia "Excelente"	Cómo lo cumple el proyecto	Verificado	En el video
Manejo correcto de secretos	Credenciales de AWS y contraseña de BD solo en GitHub Secrets; nada en el código	Búsqueda de claves en el repositorio: ninguna	4:47

Desempeño y monitoreo (30 %)

Exigencia "Excelente"	Cómo lo cumple el proyecto	Verificado	En el video
Ejecuta pruebas de carga	k6 con rampa de 23 → 150 usuarios virtuales (4 min 20 s)	Script ejecutado localmente sin errores	4:35 (inicio) y 7:00
Tiempo de respuesta, VUs, % de errores	Resumen de k6: `http_req_duration` (p95), `vus`, `http_req_failed`	Comprobado en la ejecución local	7:00 y 9:05
Saturación	`/api/stress` consume CPU; a ~100+ VUs sube la latencia	Endpoint probado	7:00 – 9:05
Funcionamiento del balanceador	`check_balancer.sh`, `instance_hits` por zona en k6, *RequestCountPerTarget*	Reparto comprobado localmente; el script tolera errores	7:15
Métricas en CloudWatch	Dashboard creado por Terraform: requests, latencia p95, 2XX/5XX, CPU, instancias del ASG, RDS	JSON del dashboard validado con `terraform test`	7:35
Relaciona métricas con la infraestructura	Frase de análisis final (CPU de las t3.micro → latencia → alarma → escalado; la BD no es cuello de botella)	—	9:05

> **Lo único que no pude probar es el despliegue real en AWS**, porque necesita sus cuentas de AWS Academy. Por eso, la sección 3 es obligatoria: deja todo funcionando y comprobado antes de grabar.

---

2. Justificaciones (para decirlas y por si el docente pregunta)

Decisión	Por qué
2 zonas de disponibilidad	Si cae una zona completa, la otra sigue atendiendo.
ALB (capa 7)	Revisa la aplicación con un health check HTTP a `/health` y reparte peticiones HTTP; un NLB no revisa la aplicación.
EC2 en subred privada	No se pueden alcanzar desde Internet; solo entra tráfico que pasa por el ALB.
Subred de BD sin ruta a Internet	La base de datos no tiene ningún camino de entrada ni de salida hacia Internet.
SG que referencian otros SG	Las reglas no dependen de IPs: cada instancia nueva del ASG queda autorizada automáticamente.

Decisión	Por qué
NAT Gateway	Las EC2 privadas descargan el código y los paquetes sin quedar expuestas.
Un solo NAT Gateway	Compromiso de costo en el lab. Solo afecta la **salida** a Internet de las EC2, no la atención a usuarios. En producción se pondría uno por zona.
Réplica de lectura	Separa lecturas de escrituras y deja una copia de los datos en otra zona.
Sin Multi-AZ en RDS	Duplica costo y tiempo en el lab. La réplica ya deja una copia en otra zona; en producción se activaría Multi-AZ para la conmutación automática.
t3.micro + escalado horizontal	Varias instancias pequeñas son más baratas y no son un punto único de falla.
Escalado por CPU al 50 %	Deja margen para absorber picos mientras arrancan las instancias nuevas (unos 3 minutos).
IMDSv2 obligatorio y RDS cifrado	Buenas prácticas de seguridad sin costo adicional.
`user_data` clona el repositorio	Cada instancia nueva arranca con la última versión del código, sin intervención manual.
Instance refresh gradual	El despliegue reemplaza las instancias de una en una, sin cortar el servicio.
Estado de Terraform local	Cada integrante despliega en su propia cuenta; en producción se usaría un backend remoto en S3.

3. Antes de grabar (obligatorio)

Material listo (ya incluido en el proyecto):

- Diagrama en PNG: `docs/arquitectura-aws.png`.
- Repositorio con todo el código.

El día anterior:

- □ El **integrante 2 crea el repositorio público** en GitHub y sube el proyecto. Debe ser el dueño, porque en un repositorio personal los colaboradores no pueden ver ni editar los *Secrets*.
- □ El integrante 2 agrega al integrante 1 como colaborador; el integrante 1 clona el repositorio.
- □ Ensayo completo cronometrado: debe durar **menos de 9:40**.

El día de la grabación, en este orden:

Hora relativa	Integrante 1 (en su cuenta)	Integrante 2 (en su cuenta)
–2:00 h	Start Lab, credenciales en `~/.aws/credentials`, `aws sts get-caller-identity`	Lo mismo
–1:55 h	`terraform apply` (25 min)	`terraform apply` (25 min)

Hora relativa	Integrante 1 (en su cuenta)	Integrante 2 (en su cuenta)
−1:25 h	Comprobar: `curl $APP/status` con primaria y réplica `ok: true`; 2 targets *healthy*	Lo mismo
−1:20 h	—	Cargar en GitHub Secrets **sus** credenciales y `DB_PASSWORD`; variable `AWS_DEPLOY_ENABLED=true`; **Run workflow** (12 min) hasta tener **4 etapas en verde**
−1:05 h	—	**Prueba k6 previa** en una pestaña de terminal que se deja abierta (ver comando abajo)
−0:15 h	Abrir pestañas (ver lista)	Comprobar que el ASG volvió a **2 instancias** (Auto Scaling Groups → *Desired capacity* = 2); abrir pestañas
0:00	Grabar	Grabar

Comando de la prueba previa (integrante 2, desde la raíz del repositorio):

```
mkdir -p loadtest/results
k6 run -e BASE_URL=$(terraform -chdir=terraform output -raw app_url) \
  -e AZS=$(terraform -chdir=terraform output -raw availability_zones) \
  --summary-export=loadtest/results/summary-previa.json loadtest/k6-loadtest.js
```

Pestañas abiertas, integrante 1: Learner Lab; diagrama PNG; consola en VPC → Subnets; consola en EC2 → Auto Scaling Groups → dgiti-ef-asg; navegador con `app_url`; editor con la carpeta `terraform/`; terminal en la raíz del repositorio.

Pestañas abiertas, integrante 2: Learner Lab; GitHub en `ci-cd.yml`; GitHub Secrets; la ejecución verde del pipeline; S3 → bucket `dgiti-ef-artifacts-…`; EC2 → Target Groups → dgiti-ef-tg; CloudWatch → dashboard `dgiti-ef-dashboard`; Auto Scaling Groups → Activity; **terminal 1** (para k6), **terminal 2** (para `check_balancer.sh`) y **terminal 3** con el resultado de la prueba previa.

Tip: suban el tamaño de letra de la terminal (Cmd/Ctrl +) y cierren notificaciones.

---

4. Integrante 1 · 0:00 – 4:30

0:00 – 0:05 · Cuenta propia

Mostrar el Learner Lab con su nombre y **AWS** ● en verde.

0:05 – 2:00 · Arquitectura

Mostrar el diagrama PNG y **decir**:

> "Usamos una VPC 10.0.0.0/16 en us-east-1 con dos zonas de disponibilidad: si cae una zona, el servicio sigue. Cada zona tiene tres capas de red: una subred pública con el balanceador y el NAT Gateway, una subred privada de aplicación con las EC2 y una subred privada de datos con RDS, sin ruta a Internet.
>
> El usuario entra por el Internet Gateway al Application Load Balancer. Usamos un ALB porque trabaja en capa 7 y revisa la aplicación con un health check a /health. El ALB reparte el tráfico entre las instancias de un Auto Scaling Group: mínimo dos, una por zona, y hasta cuatro si la CPU supera el 50 %. Si una instancia falla, el ALB deja de enviarle tráfico y el ASG la reemplaza: eso es la alta disponibilidad.
>
> La seguridad son tres Security Groups encadenados: el ALB acepta el puerto 80 desde Internet, las EC2 solo el 8080 desde el ALB y la base de datos solo el 3306 desde las EC2. Ningún servidor tiene IP pública; las EC2 salen a Internet solo por el NAT.
>
> Los datos están en RDS MySQL: la primaria recibe las escrituras y una réplica de lectura en la otra zona atiende las lecturas y guarda una copia. Elegimos varias t3.micro con escalado horizontal: es más barato y no hay un punto único de falla."

Hacer (15 s): consola → **VPC** → **Subnets**, filtrar por `dgiti-ef`:

> "Estas son las seis subredes del diagrama."

Luego **Security Groups** → **dgiti-ef-db-sg** → **Inbound rules**:

> "La base de datos solo acepta al Security Group de la aplicación."

2:00 – 2:40 · Servicio desplegado

1. Navegador: abrir `app_url` y recargar 3 o 4 veces.
   > "Cada recarga la atiende otra instancia, en otra zona: el balanceador está repartiendo."
2. Terminal (desde la raíz del repositorio):

```
APP=$(terraform -chdir=terraform output -raw app_url)
curl -s $APP/health
curl -s $APP/status
curl -s -X POST $APP/api/visits
curl -s $APP/api/visits
```

> "/health es el endpoint de validación del balanceador. /status confirma la conexión con la primaria y con la réplica. El POST escribe en la primaria y el GET lee de la réplica: dice read_from replica."

2:40 – 4:08 · Infraestructura como Código

Lanzar `init` y `plan`, y **explicar los archivos mientras `plan` consulta AWS**:

```
cd terraform
terraform init
terraform plan
```

> "provider.tf define el proveedor AWS, sin credenciales en el código. variables.tf parametriza la red, el ASG, el tipo de instancia y la base de datos; la contraseña es sensible y tiene validación. main.tf crea la red, los Security Groups, RDS con su réplica, S3, el ALB, el Auto Scaling Group y el dashboard de CloudWatch. user_data.sh es lo que ejecuta cada EC2 al arrancar: clona el repositorio y levanta la aplicación. outputs.tf entrega las URLs. El tfvars real no se sube a Git. Todo lo crea Terraform, 46 recursos; nada está hecho a mano."

Cuando termine el plan:

> "No changes: lo desplegado coincide exactamente con el código."

```
terraform apply -var="app_version=1.1.0"
```

Mientras muestra el plan, decir:

> "Cambio la versión de la aplicación: Terraform crea una nueva versión del Launch Template y el ASG reemplaza las instancias de una en una."

Escribir `yes`. Al ver *Apply complete*:

```
terraform output
```

> "Los outputs: la URL del servicio, los endpoints de la base de datos y el dashboard."

4:08 – 4:30 · Alta disponibilidad en vivo

```
../loadtest/check_balancer.sh $(terraform output -raw app_url) 20
```

Mostrar **Auto Scaling Groups** → **dgiti-ef-asg** → **Instance refresh** (en curso) y decir:

> "Ahora mismo el ASG está sacando de servicio una instancia para reemplazarla, y las 20 solicitudes responden sin errores: el servicio sigue disponible aunque una instancia salga. Le paso a mi compañero."

Después de grabar: ponga `app_version = "1.1.0"` en su `terraform.tfvars`, para que el próximo `plan` no muestre cambios.

---

5. Integrante 2 · 4:30 – 10:00

4:30 – 4:35 · Cuenta propia

Mostrar el Learner Lab con su nombre y **AWS ●** en verde.

4:35 – 4:47 · Lanzar la prueba de carga

En la **terminal 1**, desde la raíz del repositorio:

```
k6 run -e BASE_URL=$(terraform -chdir=terraform output -raw app_url) \
  -e AZS=$(terraform -chdir=terraform output -raw availability_zones) \
  --summary-export=loadtest/results/summary.json loadtest/k6-loadtest.js
```

> "Lanzo la prueba de carga contra mi balanceador; dura cuatro minutos y vuelvo a ella al final."

4:47 – 7:00 · Pipeline CI/CD

Con `.github/workflows/ci-cd.yml` abierto, **decir**:

> "El pipeline corre con cada push a main y tiene cuatro etapas. La etapa uno valida el código con flake8 y ejecuta quince pruebas unitarias con pytest. La etapa dos valida Terraform con fmt y validate. Si ambas pasan, la etapa tres prepara el despliegue: se autentica en mi cuenta de AWS Academy con secretos de GitHub y ejecuta terraform plan. La etapa cuatro despliega: hace un instance refresh del Auto Scaling Group para que las instancias nuevas tomen el último commit, corre pruebas de integración contra el balanceador real y guarda los reportes en S3."

Hacer:

1. **Settings** → **Secrets and variables** → **Actions**:
   > "Las credenciales están solo como secretos; aquí se ven los nombres, nunca los valores, y no hay ninguna clave en el código."
2. Abrir la **ejecución en verde** hecha antes de grabar:
   - Etapa 1 → *Pruebas unitarias con cobertura*: señalar `15 passed`.
   - Etapa 2 → *terraform validate*: señalar `Success! The configuration is valid.`
   - Etapa 3 → *terraform plan*:
     > "El plan del pipeline no tiene el estado local, por eso lista los 46 recursos: valida credenciales, proveedor y configuración antes de desplegar."
   - Etapa 4 → *Desplegar*: señalar `estado=Successful`. Luego *Pruebas de integración*: señalar `4 passed`.
     > "Incluida la prueba que verifica que responden al menos dos instancias distintas."
3. **S3** → **dgiti-ef-artifacts-…** → **ci/** (ejecución más reciente):

> "Aquí están los reportes que subió el pipeline."

> ⚠️ **No hagan push a `main` ni usen "Run workflow" durante la grabación.** La etapa 4 reemplazaría instancias en medio de la prueba de carga.

7:00 – 10:00 · Desempeño y monitoreo

1. **Terminal 1, k6 corriendo (15 s):**

> "k6 está subiendo hacia 150 usuarios virtuales. Cada uno llama a /api/info y a /api/stress, que consume CPU para llevar el sistema a saturación."

2. **Terminal 2 (15 s):**

```
./loadtest/check_balancer.sh $(terraform -chdir=terraform output -raw app_url) 20
```

> "Las respuestas vienen de instancias en las dos zonas: el ALB reparte la carga."

3. **EC2 → Target Groups → dgiti-ef-tg (20 s):** pestaña *Targets* y luego *Monitoring*.

> "Todas las instancias están healthy y las solicitudes al target group están subiendo."

4. **CloudWatch → dgiti-ef-dashboard (60 s)**, con el rango en **3h** para que se vean la prueba previa y la actual:

> "En solicitudes por minuto se ven los picos de las pruebas. El tiempo de respuesta p95 sube cuando pasamos de unos cien usuarios: ese es el punto de saturación. En códigos HTTP vemos los 2XX y si aparece algún 5XX. Solicitudes por instancia muestra que el tráfico se divide entre los targets. La CPU del ASG supera el objetivo del 50 %, y por eso las instancias en servicio pasan de dos a tres o cuatro. En RDS la CPU queda baja y el retraso de la réplica es mínimo: la base de datos no es el cuello de botella."

5. **Auto Scaling Groups → Activity (15 s):**

> "Aquí está el evento: el Auto Scaling lanzó instancias nuevas por la alarma de CPU."

6. **Terminal 1 con el resumen de k6 (50 s).** Si no terminó, usar la **terminal 3** con la prueba previa. Leer los valores reales:

> "El tiempo de respuesta p95 fue de [valor de http_req_duration p95], con [valor de http_req_failed] de errores y un máximo de 150 usuarios virtuales. instance_hits muestra que la zona A atendió [número] solicitudes y la zona B [número]: el balanceador repartió de forma equilibrada. En resumen: al saturar, la latencia subió porque la CPU de las t3.micro llegó a su límite; CloudWatch lo registró, la alarma de CPU disparó el Auto Scaling y las instancias nuevas repartieron la carga. Por eso diseñamos con balanceador y escalado horizontal."

Si k6 marca ✗ en `http_req_duration`, agregar:

> "Ese umbral de 1,5 segundos se superó en la fase de 150 usuarios: es la evidencia de saturación que buscábamos."

7. **Cierre (5 s):**

> "Infraestructura automatizada con Terraform, desplegada con CI/CD, validada bajo carga y monitoreada en CloudWatch. Gracias."

---

6. Si algo falla durante la grabación

Problema	Qué hacer
`ExpiredToken` en Terraform	Las credenciales vencieron: AWS Details → copiar de nuevo → repetir el comando. Comprueben con `aws sts get-caller-identity` justo antes de grabar.
`plan` muestra cambios en vez de *No changes*	Decir "Terraform detecta diferencias y las corrige" y continuar con el `apply`.
La prueba k6 no termina a tiempo	Mostrar la terminal 3 con la prueba previa y leer esos valores.
El ASG no escaló en vivo	Con el dashboard en 3h se ve el escalado de la prueba previa; en *Activity* aparecen sus eventos.
`check_balancer.sh` muestra alguna línea `ERROR`	Explicarlo: "son las solicitudes de la instancia que salía de servicio; el ALB dejó de enviarle tráfico".
Un target aparece *unhealthy*	Es una instancia nueva que todavía está arrancando (unos 3 minutos); el ALB no le envía tráfico.

---

7. Después de grabar

Los dos ejecutan `terraform destroy` en su cuenta para no consumir el presupuesto del lab.